计算机网络实验报告：协议数据单元的封装与解封装

实验名称：基于 Ethernet/IP/UDP 的数据包封装与解封装 **姓名**：段雯菀 **学号**：20231326 **日期**：2025年12月20日

---

1. 实验目的

1. **理解 SDN/OpenFlow 架构**：深入理解软件定义网络（SDN）中控制平面（Control Plane）与数据平面（Data Plane）分离的核心思想。
2. **掌握流表（Flow Table）机制**：通过代码模拟，掌握 OpenFlow 交换机如何基于流表（Flow Table）进行数据包的匹配（Match）与动作执行（Action）。
3. **模拟 Type 0 交换机**：实现 OpenFlow 白皮书中定义的 "Type 0 Switch" 的基本功能，包括 10 元组特征提取和基本动作（Forward, Drop, Packet-In）。

2. 实验原理

本实验基于 *Nick McKeown* 等人发表的论文 **"OpenFlow: Enabling Innovation in Campus Networks"**。
OpenFlow 交换机不再使用传统的二/三层目的地址路由表，而是使用**流表（Flow Table）**来处理数据包。

2.1 核心组件

- **流表 (Flow Table)**：包含多个流表项（Flow Entries），每个表项由 **包头域（Header Fields）**、**计数器（Counters）** 和 **动作（Actions）** 组成。
- **安全通道 (Secure Channel)**：连接交换机与远程控制器（Controller）的接口。
- **OpenFlow 协议**：控制器用于修改交换机流表的标准协议。

2.2 处理流程

1. **解析 (Parsing)**：提取数据包的头部信息（如 MAC 地址、IP 地址、TCP/UDP 端口等）形成 **Flow Key**。
2. **查表 (Lookup)**：将 Flow Key 与流表中的规则进行匹配。
3. **执行动作 (Action)**：
    - **Forward**：转发到指定端口。
    - **Drop**：丢弃数据包。
    - **Encapsulate to Controller**：封装并发送给控制器（通常用于首包）。

---

3. 关键代码实现

本实验使用 C 语言模拟了一个 OpenFlow 交换机的数据面逻辑。

3.1 核心数据结构设计

为了模拟 OpenFlow 论文中的 **Type 0 Switch**，定义了包含 10 个关键匹配域的结构体：

```
// [OpenFlow 论文核心] 10元组匹配域 (10-tuple Match Fields)
typedef struct {
    int ingress_port;           // 入端口
    unsigned char src_mac[6];   // 以太网源地址
    unsigned char dst_mac[6];   // 以太网目的地址
    unsigned short eth_type;    // 以太网类型
    unsigned short vlan_id;     // VLAN ID
    unsigned char vlan_prio;    // VLAN 优先级
    unsigned char src_ip[4];    // IP 源地址
    unsigned char dst_ip[4];    // IP 目的地址
    unsigned char ip_proto;     // IP 协议号 (TCP/UDP/ICMP)
    unsigned short src_port;    // 传输层源端口
    unsigned short dst_port;    // 传输层目的端口
} OpenFlowMatchKey;
```

3.2 包解析模块 (Parser)

不同于传统解封装，该模块一次性提取跨层级的关键信息：

```
void parser_extract_headers(unsigned char* buffer, int total_len, OpenFlowMatchKey* key) {
    // 1. 提取以太网头 (EtherType)
    // ...
    key->eth_type = eth.type;

    // 2. 提取 IP 头 (Protocol, SrcIP, DstIP)
    if (key->eth_type == 0x0800) {
        // ...
        key->ip_proto = ip.protocol;

        // 3. 提取 UDP 头 (SrcPort, DstPort)
        if (key->ip_proto == 17) {
            // ...
            key->dst_port = udp.dst_port; // 关键匹配字段
        }
    }
}
```

3.3 查表与转发逻辑 (Lookup & Action)

模拟 TCAM 查找过程，遍历流表进行匹配，并执行动作：

```
void openflow_switch_process(unsigned char* buffer, int total_len) {
    // 1. 解析 (Parser)
    OpenFlowMatchKey key;
    parser_extract_headers(buffer, total_len, &key);

    // 2. 查表 (Lookup)
```

```
    FlowTableEntry* entry = lookup_flow_table(&key);

    // 3. 执行动作 (Action)
    if (entry != NULL) {
        if (entry->action == ACTION_FORWARD) {
            printf("[Action] FORWARD packet to Port %d\n", entry->out_port);
        }
        // ... 其他动作处理
    }
}
```

4. 实验结果与分析

4.1 控制台运行输出

实验运行结果截图如下：

```
Active code page: 65001
PS C:\Users\33219> cd "e:\专业课\大三上\计算机网络\计网作业\" ; if ($?) { gcc project.c -o project } ; if ($?) { .\project }
=== [Host A] Generating Traffic ===
Packet sent to wire (testmsg.txt).

=== [OpenFlow Switch] Booting Up ===
[Controller] Flow Table Initialized. Rule #0 added: Match UDP DstPort 8091 -> Action: Forward Port 2
Packet Received on Ingress Port 1. Length: 59 bytes

=== [OpenFlow Switch] Processing Packet ===
Parsed Flow Key:
  EtherType: 0x0800
  IP Proto:  17
  L4 Ports:  Src 8090 -> Dst 8091
Match Found in Flow Table!
[Action] FORWARD packet to Port 2 (Line-rate)
```

4.2 结果详细分析

1. **控制平面下发 (Flow Mod)**：
    - 输出 `[Controller] Flow Table Initialized... Rule #0` 表明模拟的控制器成功向交换机下发了一条规则：**匹配 UDP 目的端口 8091 的数据包，将其转发到端口 2**。这是 SDN 的核心特征。
2. **数据包解析 (Parsing)**：
    - 输出 `Parsed Flow Key` 显示交换机正确识别了数据包特征：
        - `EtherType: 0x0800` (IPv4)
        - `IP Proto: 17` (UDP)
        - `Dst: 8091`
    - 这证明了解析模块（Parser）工作正常，能够从二进制流中提取 10 元组信息。
3. **流表匹配 (Matching)**：
    - 输出 `Match Found in Flow Table!` 表明提取的 Flow Key 与流表中的 `Rule #0` 成功匹配。程序正确地通过对比 `Dst Port` 字段完成了查找。
4. **动作执行 (Forwarding)**：
    - 输出 `[Action] FORWARD packet to Port 2` 验证了交换机执行了流表中定义的指令。

5. 实验总结

通过本次实验，我成功地从代码层面模拟了 OpenFlow 协议的工作机制。与传统网络实验相比，本次实验有以下几点重要收获：

1. **视角转换**：从传统的“按层解包”（Decapsulation）转向了扁平化的“流特征提取”（Flow Extraction）。交换机不再关心数据包是 HTTP 还是 FTP，只关心它是否匹配流表中的特征。
2. **控制分离**：通过代码中的 `controller_init` 和 `switch_process` 两个函数的划分，直观地理解了控制平面（制定规则）和数据平面（执行规则）的分离。
3. **灵活性**：我认识到通过修改流表（Flow Table），可以在不修改交换机硬件代码的情况下改变网络的行为（例如将转发改为丢弃），这正是 SDN “软件定义”优势的体现。

实验代码运行稳定，逻辑符合 OpenFlow 白皮书规范，达到了预期目标。